

Using Multivariate Polynomial Linear Regression to Model Asymmetric Lift Kinematics

Oliver Greenwood

Peters' Research[3], Great Missenden UK

11th of September 2024

Abstract

Efficient modeling of lift kinematics is vital for improving elevator systems to enhancing passenger comfort and run accurate simulations. Current methods involve equations that produce an estimate for the ideal kinematics. This study aims to evaluate the effectiveness of polynomial regression models in predicting lift kinematics and compare different methods for training the model. Accelerometer data was collected from 104 lift journeys, encompassing approximately 156,000 data points. Using multivariate polynomial linear regression with varying degrees, we developed models to predict key kinematic parameters: acceleration, velocity, jerk, and displacement. Model performance was assessed using cross-validation and root mean square error (RMSE). Using our initial method, the models was able to predict value with high R^2 scores for velocity, acceleration, and displacement, all exceeding 90%. The method also yielded good RMSE values, approximately 0.08 for both velocity and acceleration, and 1.8 for displacement. The model produced predicted asymmetric ideal kinematics with two different jerk values. The second method was specifically designed to optimise the model for predicting acceleration. With this refined approach, the predictions obtained had a significantly improved R^2 score of 99% and an RMSE of 0.03, demonstrating enhanced model accuracy for acceleration predictions. This method also allowed for four different jerk values. Polynomial linear regression proves to be a robust tool for modeling complex lift kinematics. These findings have implications for more accurate lift simulations and improved passenger experience.

Keywords: kinematics, lift, elevator, asymmetric profile, lift kinematics, twin lift system, 2 cars per shaft, multi-dimensional lift system, simulation, machine learning, polynomial regression

1 INTRODUCTION

Lift kinematics plays a crucial role in the elevator industry, focusing on the movement of a lift car within a shaft without considering mass or force. By measuring accelerometer data from a lift, we can assess the smoothness of its motion and improve passenger comfort based on this analysis. Additionally, this data can help detect potential issues in the lift system before they affect the user experience. This paper presents a method for generating equations for use in both lift simulations and real-world lift systems.

1.1 Definitions

Table 1: Definitions of Key Terms in Lift Kinematics

Term	Definition
Lift Kinematics	The study of the motion of a lift car in a shaft, without considering mass or force.
Jerk	The rate of change of acceleration with respect to time. It is the derivative of acceleration and measures how smoothly a lift starts or stops.
Acceleration	The rate at which the velocity of the lift changes with time.
Velocity	The speed of the lift in a given direction. It is the derivative of displacement with respect to time.
displacement	The vertical displacement of the lift car from a reference point, typically the floor of the shaft.
Polynomial Regression	A form of regression analysis in which the relationship between the independent variable and the dependent variable is modeled as an nth degree polynomial.
Phase	A segment of displacement, velocity, acceleration, or jerk data where the acceleration is continuously not zero.
Period	A segment of displacement, velocity, acceleration, or jerk data where the jerk is constant.
Single Journey	A journey where the lift goes from one floor to another.
R^2	A statistic of the model that shows how well the model can predict the variance of the data.
Root Mean Square Error (RMSE)	A statistic of the model that shows the average difference between the prediction and the actual values.
M_{acc}	An array of data points representing acceleration values, where each acceleration value corresponds to a specific timestamp given in a respective timestamp array.
M_{vel}	An array of data points representing velocity values, where each velocity value corresponds to a specific timestamp given in a respective timestamp array.
M_{jerk}	An array of data points representing jerk values, where each jerk value corresponds to a specific timestamp given in a respective timestamp array.
$M_{displacement}$	An array of data points representing displacement values, where each displacement value corresponds to a specific timestamp given in a respective timestamp array.
j	The maximum jerk in a given segment.
a	The maximum acceleration in a given segment.
v	The maximum velocity in a given segment.
d	The final displacement in a given segment.

2 METHODOLOGY

2.1 Data Collection

The machine learning models requires a large dataset for training and evaluating the model. For this model, 104 motion files (.mtn) were collected, each containing raw accelerometer data and timestamps from single journeys in a csv format. These files also contain 3 values at the top representing stages of the door cycle on a lift, these can be ignored. Each file contained approximately 25 seconds of data, and with 60 data points per second, about 156,000 unfiltered data points were available for training the model.

2.2 Processing

As the data from the motion files is raw accelerometer data in its x, y and z components, it must be processed into a single smoothed set of data, which can then be used with the timestamps to calculate the corresponding velocity, jerk and displacement values.

2.2.1 Acceleration

Pythagoras' theorem is applied to the x, y, and z components of the acceleration to calculate the magnitude, and it is multiplied by the sign of the z-component to allow for different orientations of the accelerometer.

$$M_{acc} = \sqrt{(\text{accelerometer}_x)^2 + (\text{accelerometer}_y)^2 + (\text{accelerometer}_z)^2} \times \text{sign}(\text{accelerometer}_z) \quad (1)$$

Smoothing is now applied to M_{acc} . This will remove a considerable amount of the noise from the data. The following function is used:

$$M_{acc_i} = \frac{M_{acc_i} + M_{acc_{i-1}} \cdot \alpha}{1 + \alpha} \quad (2)$$

The differentiating that is applied later will greatly exaggerate the noise from the data. This means a very small alpha value must be applied to reduce the noise considerably. A value of 0.025 is used as it is sufficiently small but not so small that it renders the data unusable.

Finally, accelerometer drift is corrected using this algorithm:

Listing 1: Accelerometer Drift Correction

```
rolling_sum <- 0
```

```

num_of_pnts <- 0
for acc in M_acc {
  if (
    acc is first in M_acc or
    abs(rolling_sum / num_of_pnts - acc) < bound
  ) {
    rolling_sum += acc
    num_of_pnts++
  }
}
M_acc == rolling_sum / num_of_pnts

```

2.2.2 Assigning Stages

Stages are needed to be defined at each timestamp so that the behaviour the lift is exhibiting in any given instance of time is known. This will be useful in measurement corrections and splitting the data into phases and periods.

Table 2: Stages of Lift Motion

Stage	Description
0	Lift is stationary
1	Lift is accelerating
2	Lift is at constant velocity
3	Lift is decelerating

These stages are assigned using a similar algorithm to this dissertation [2]:

First, an upper and lower threshold for acceleration is defined. A lower bound of 0.1 was used as it was as low as possible whilst still catching all the 0 acceleration values and accounting for the noise. A upper bound of 0.5 was used as it was as high as possible whilst sill catching all the peak acceleration values.

$$a^{UB} = a * UB \quad (3)$$

$$a^{LB} = a * LB \quad (4)$$

M_{acc} was looped through and simultaneously C was calculated (the number of consecutive instances we find above a^{UB}). If C ever satisfies:

$$C \geq C_{lim} \quad (5)$$

Then the algorithm immediately backtracks and determines where a^{LB} was crossed and set all points past that to stage one. A similar process to this is executed for converting stage one to stage two, stage two to stage three and stage three to stage zero.

2.2.3 Velocities

The M_{acc} is integrated with respect to time to compute the M_{vec} . Corrections are applied on M_{vel} using the stage values calculated previously. When at stage 0, the velocity is drawn to the value it should be in increments, similar to this dissertation [2].

Listing 2: Measured Velocity Correction Algorithm

```
for index in range(0, M_vel.length):  
    if (stage at index) == 0 {  
        if M_vel[i] > effect {  
            M_vel[i:] -= effect  
        } else if M_vel[i] < - effect {  
            m_vel[i:] += effect  
        } else {  
            m_vel[i] = 0  
        }  
    }
```

The effect variable was found through trial and error. For this model, 0.0005 was used. Higher values would result in velocity having sudden changes when changing to stage zero. Lower values would result in velocity not changing sufficiently.

2.2.4 displacement and Jerk

The velocity is integrated to compute the $M_{displacement}$ (displacement), and the acceleration is differentiated to compute the M_{jerk} (jerk). No corrections are required for these values.

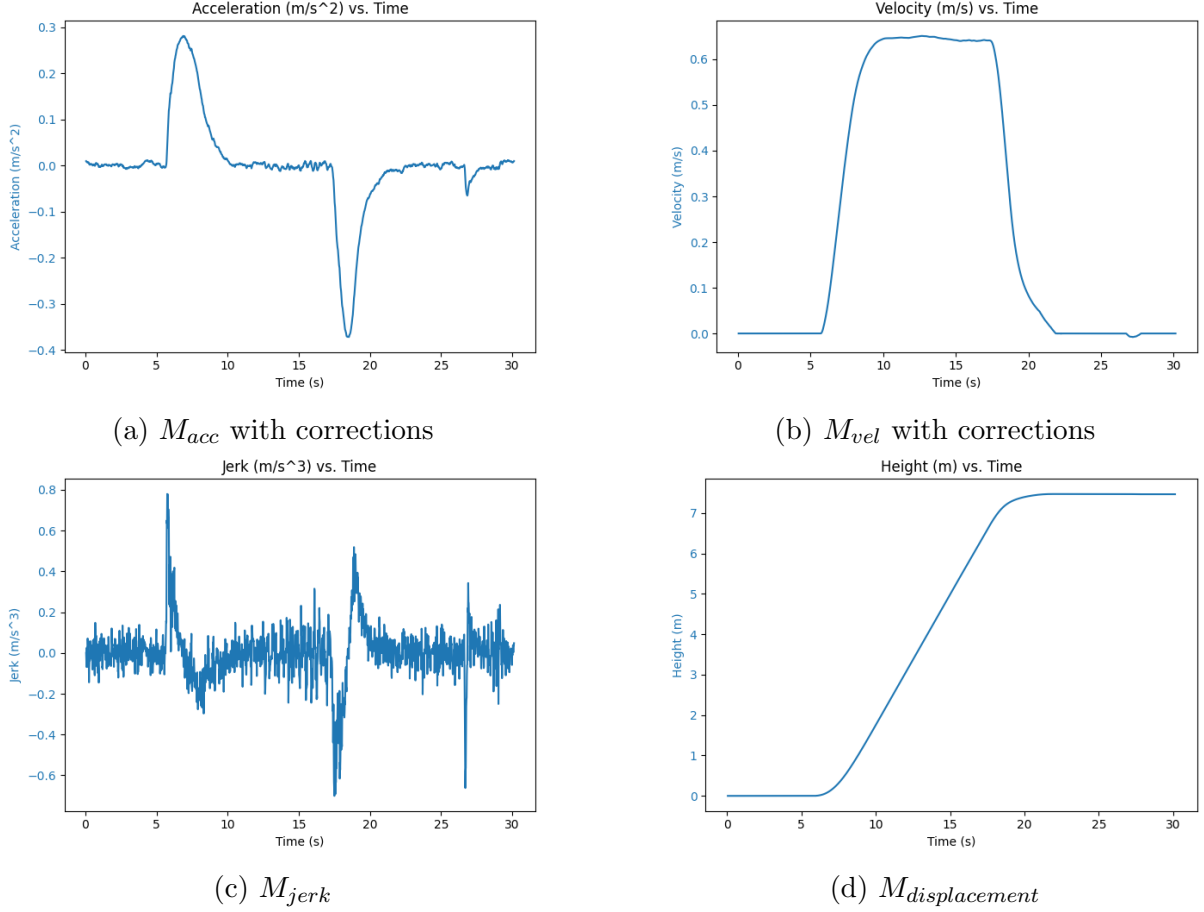


Figure 1: Graphical illustrations of measured values with corrections

2.3 Method 1: Using Phases

2.3.1 Splitting the Data

The phases are extracted from this data with a buffer zone. This buffer zone is to help the linear regression learn the start and end values of the phase. For each phase, j , a , v , and d is calculated and saved. The phases are then translated and transformed so they all start at time 0 at value 0.

2.3.2 Building the Polynomial Regression Model

The model was developed using Scikit-Learn, an open-source Python library for machine learning. Initially, the data was divided into training and testing datasets, with 90% allocated for training and 10% reserved for testing.

The model was constructed using the following pipeline:

Listing 3: Defining the Model

```
poly_model = make_pipeline(
    PolynomialFeatures(
        degree,
        include_bias=False
```

) , Ridge(alpha=0.85)
)

Here, the `include_bias` flag is set to `False` to indicate that a y-intercept of zero is required. Ridge regression, also known as L2 regularization, is employed to mitigate the risk of overfitting by reducing model complexity and controlling large coefficients. For further details, see the Ridge Regression documentation provided by IBM [5].

The degree of the polynomial features was initially estimated, but it was optimised through iterative testing with various degrees to determine the most effective configuration.

Training features were prepared by constructing a pandas DataFrame [4]. This DataFrame included a column of timestamps and four additional columns representing jerk (j), acceleration (a), velocity (v), and displacement (d), depending on the specific phase each timestamp represented.

Cross-validation was performed with 5 splits to evaluate model performance. The RMSE score was calculated to assess the model's accuracy. Following training, the model was tested, and the results were analysed and presented.

2.3.3 Adjusting the Model

Currently, the model can train to predict any of M_{acc} , M_{vel} , M_{jerk} , and $M_{displacement}$. The model was trained with degrees from 2 to 6 to find the optimal degree for each kinematics profile. Degrees any higher than this would lead to the model overfitting and producing unusable results. These were the results.

Table 3: Table Degree Key

Degree	Colour
2	Blue
3	Green
4	Red
5	Yellow
6	Purple

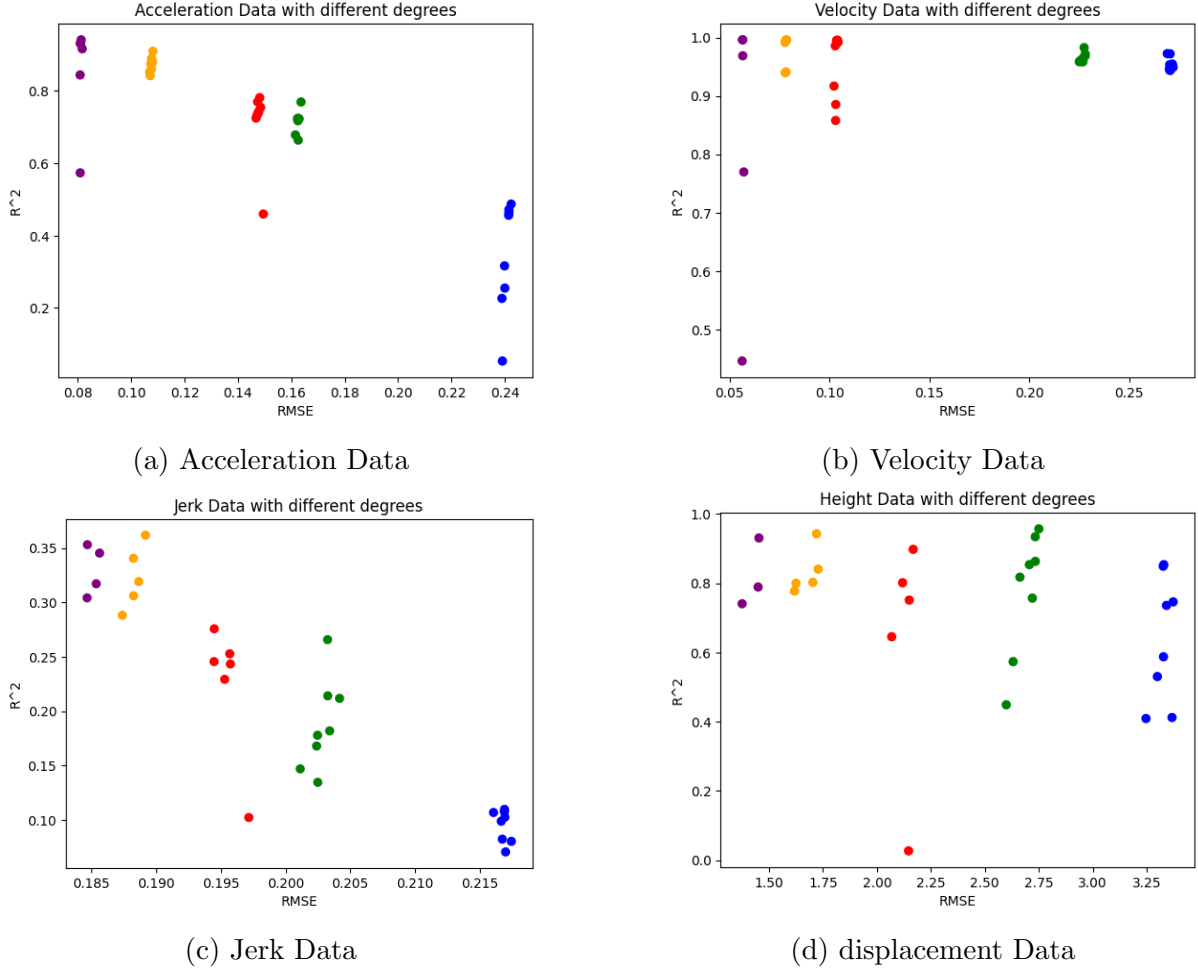


Figure 2: Results of different degrees

From this data, a degree of 6 for the models training on displacement data, acceleration data and velocity data as it gave the highest accuracy with a high consistency. The model training on jerk data will no longer be used as the data is too noisy to produce any useful results.

The model was run with these chosen degrees.

Table 4: Model Performance Metrics for Different Variables

Variable	R^2 Score	Mean RMSE Score
M_{acc}	0.913	0.094
M_{vel}	0.996	0.078
$M_{\text{displacement}}$	0.935	1.742

From this, the velocity model was chosen to be the main model which could then differentiated/integrated to find the other variables.

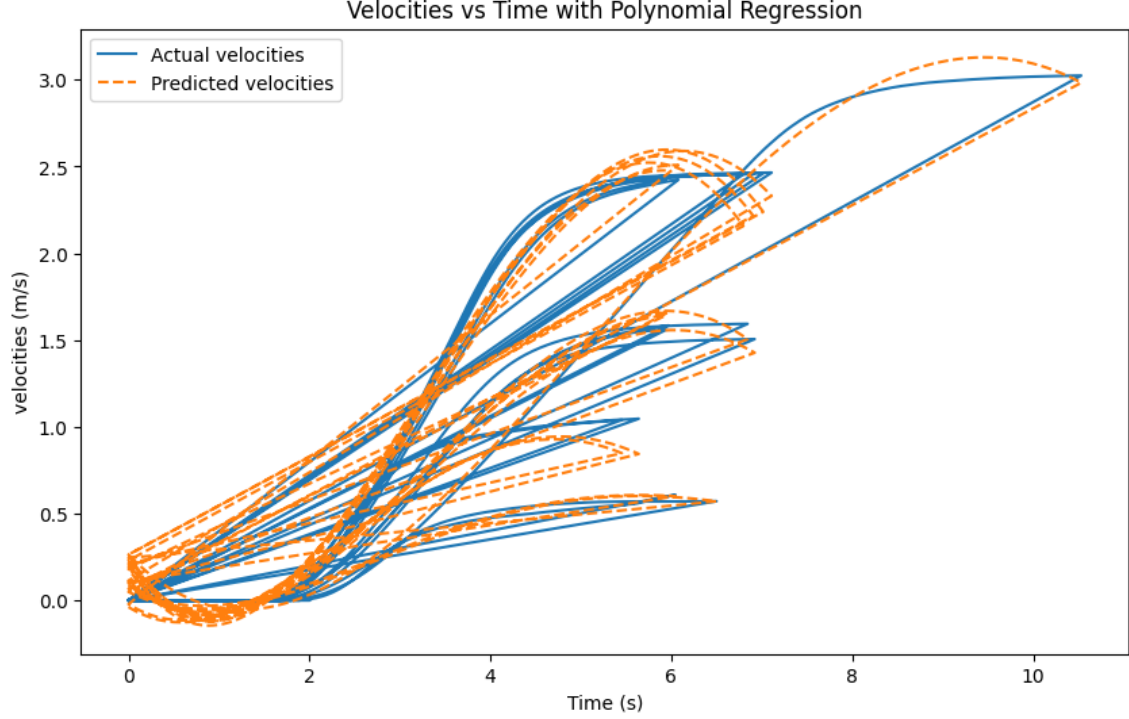


Figure 3: Velocity predicted and Velocity measured against time

2.3.4 Using the Model

To use the model, the phases (with the same size buffer zone) must be extracted, and the j , a , v , and d for that phase must be calculated. These values along with all the timestamps are inputted to the model and the predicted velocity is returned.

2.3.4.1 Calculating Ideal Acceleration

The velocities are differentiated to get the ideal accelerations. The ideal accelerations are then scaled to match the M_{acc} and then we cut off any accelerations that are in the wrong direction.

2.3.4.2 Calculating Ideal Velocity

For a more accurate ideal velocity, the ideal accelerations are integrated then scaled to v to match the M_{vel}

2.3.4.3 Calculating Ideal displacements

To calculate the displacement, the ideal velocity is integrated and a small linear correction is added.

2.3.4.4 Calculating Ideal Jerk

The ideal accelerations are differentiated to calculate the ideal jerk values. These values are then smoothed to reduce the noise.

2.4 Method 2: Using Periods

In this method, the focus is on optimising the model's acceleration profile predictions. The model is allowed to use different acceleration and deceleration values, this will allow the model to be used in scenarios such as in a two car shaft.

2.4.1 Splitting the Data

To split the data into periods, the data is split into phases like in method one but then further split them by finding when $0.9 * a$ is reached and then stopping just before the acceleration goes down again. (To find the other period, simply reverse the data and perform the same method). Small buffer zones are added to each of the periods to help the model learn to start and end on the correct values. Each period then has its a , j , v and d calculated.

2.4.2 Building the Polynomial Regression Model

The model was developed using Scikit-Learn [1], an open source Python [6] library for machine learning. Initially, the data was divided into training and testing datasets, with 90% allocated for training and 10% reserved for testing.

The model was constructed using the following pipeline:

Listing 4: Defining the Model

```
poly_model = make_pipeline(  
    PolynomialFeatures(  
        degree ,  
        include_bias=False  
    ) , Ridge(alpha=0.85)  
)
```

Here, the `include_bias` flag is set to `False` to indicate that a y-intercept of zero is required. Ridge regression, also known as L2 regularization, is employed to mitigate the risk of overfitting by reducing model complexity and controlling large coefficients. For further details, see the Ridge Regression documentation provided by IBM [5].

The degree of the polynomial features was initially estimated, but it was optimised through iterative testing with various degrees to determine the most effective configuration.

Training features were prepared by constructing a pandas DataFrame[4]. This DataFrame included a column of timestamps and four additional columns representing jerk (j), acceleration (a), velocity (v), and displacement (d), depending on the specific phase each timestamp represented.

Cross-validation was performed with 5 splits to evaluate model performance. The RMSE score was calculated to assess the model's accuracy. Following training, the model was tested, and the results were analysed and presented.

2.4.3 Adjusting the Model

Table 5: Table Degree Key

Degree	Colour
4	Blue
5	Green
6	Red
7	Yellow
8	Purple

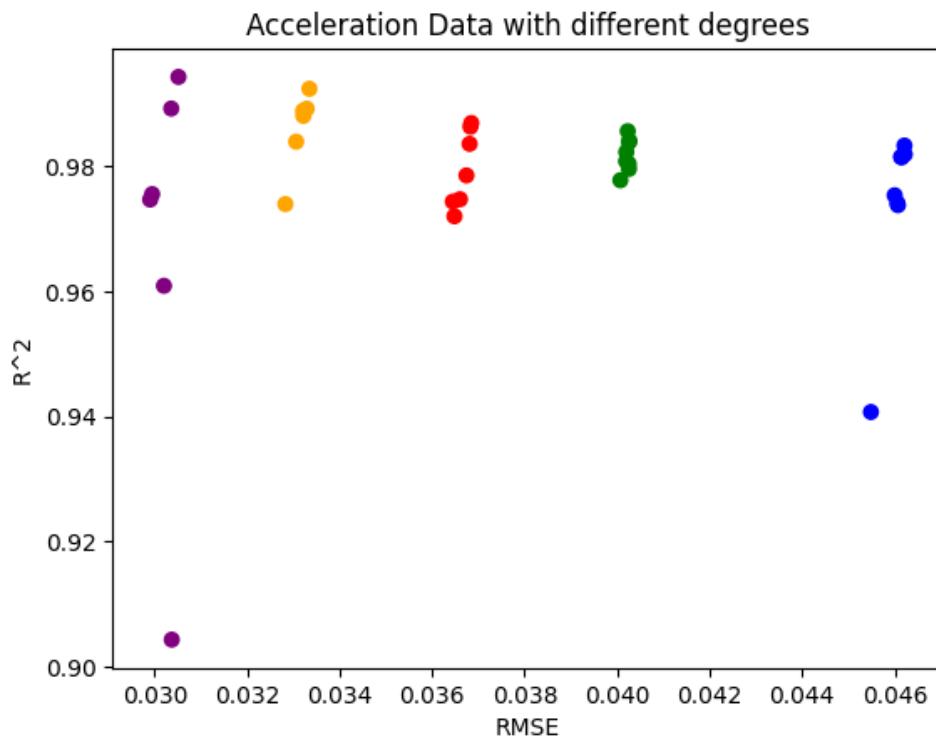


Figure 4: Acceleration predicted using models of different degrees.

From these results, it is determined that as the degree increases, the higher the chance of overfitting. For the rest of this method, a degree of 7 is used as it is consistently not overfitting and is able to produce an R^2 score of up to 0.992 and an RMSE score of as low as 0.032.

Here are the test results from a model with degree 7.

2.4.4 Using the model

To use the model, the periods (with the same size buffer zone) must be extracted, and the j , a , v , and d for that period must be calculated. These values along with all the timestamps are inputted to the model and the predicted acceleration is returned.

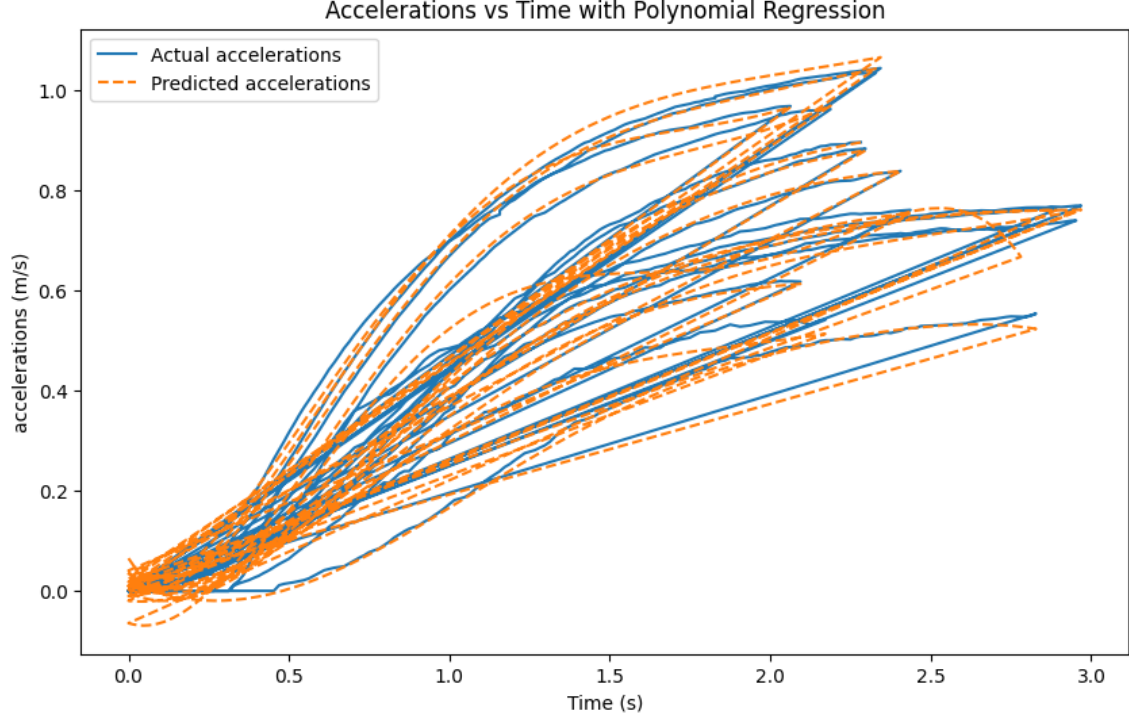


Figure 5: Acceleration predicted and Acceleration measured against time

2.4.4.1 Calculating Ideal Acceleration

First, each period is transformed so that it is in the correct orientation, then it is translated so that it starts at the correct y coordinate. Any values that are on the wrong side of $y = 0$ are removed. Any missing timestamps are removed and then the periods are connected together. Finally the acceleration is smoothed so that there are no jagged jerk values later on.

2.4.4.2 Calculating Ideal Velocity

To calculate velocity, the predicted acceleration values are integrated and scaled so the maximum predicted velocity matches v .

2.4.4.3 Calculating Ideal displacements

The ideal velocity are integrated and a small linear correction is added.

2.4.4.4 Calculating Ideal Jerk

The ideal accelerations are smoothed using an exponential weighted average, and then smoothed using equation (2). This prevents the ideal jerk values having too much noise. Finally, the ideal accelerations are differentiated to calculate the ideal jerk.

3 RESULTS

3.1 Method One

Method one demonstrated strong performance in capturing the variance in acceleration, velocity, and displacement. It also achieved low RMSE values for both acceleration and velocity. However, it struggled to produce an accurate RMSE value for displacement. Here are the results from training a model to predict velocity and then integrated and differentiated to find the other kinematic profiles.

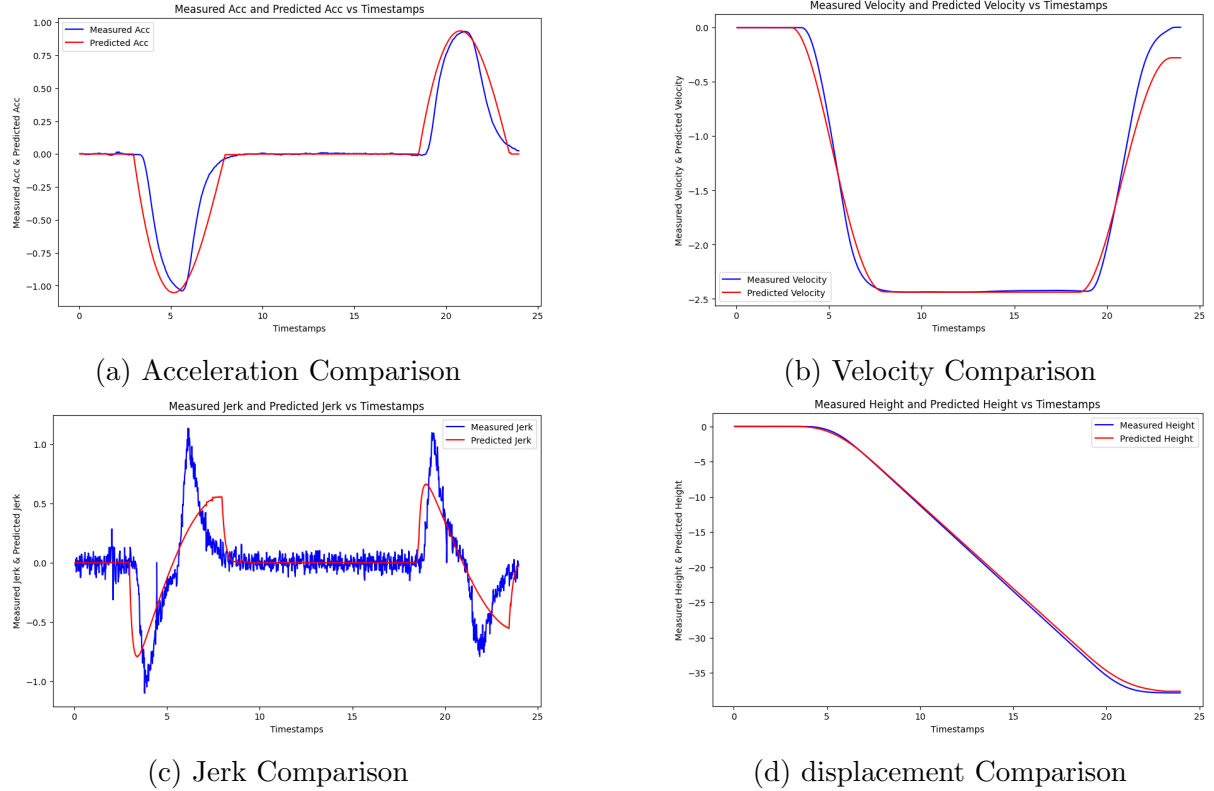


Figure 6: Results of measured data vs predicted data using method one

3.2 Method Two

Method two was designed as an optimisation for primarily predicting accelerations. This method also allowed four different jerk values. The R^2 value was 99.2% and the RMSE value was 0.03. Here are the results from training a model to predict accelerations and then using those acceleration values to calculate the other kinematic profiles.

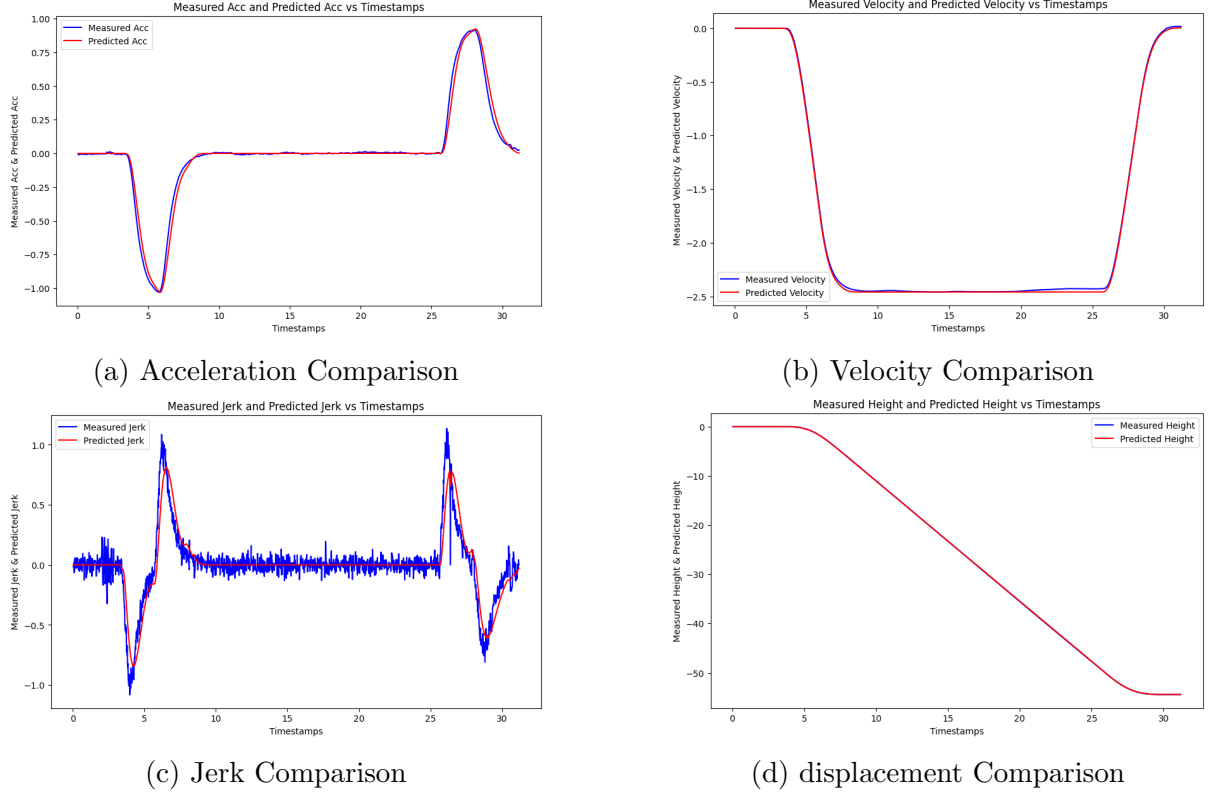


Figure 7: Results of measured data vs predicted data using method two

4 DISCUSSION

Although both methods are accurate at predicting the kinematic profiles, method two is much more accurate, offering half the RMSE. Method two also allows for a more advanced asymmetric kinematic profiles where four different jerk values can be provided unlike method two in which only two jerk values can be provided.

4.1 Further Improvements:

Currently, the only data being used is from the accelerometer. This means the model is only predicting what the accelerometer is sensing which for large displacements may be wrong. Adding another sensor and using a Kalman filter would provide results which are more accurate to the real world.

This paper only attempts to model single journeys, however to allow the model to predict multiple journeys would not be too difficult, only changing corrections towards displacement and some other minor edits.

Finally, to model dynamic kinematics, an edit to how the stages work would be required.

5 CONCLUSION

This paper shows that polynomial regression can effectively model lift kinematics, offering a valuable tool for improving simulation accuracy and passenger comfort. Whilst the

results are promising, the study's limitations highlight the need for further research with larger datasets. Overall, the findings contribute to the ongoing development of modeling techniques in the elevator industry .

References

- [1] Alexandre Gramfort Fabian Pedregosa, Gael Varoquaux and Vincent Michel. Scikit learn. <https://scikit-learn.org/stable/index.html>, 2010. Accessed: 2024-09-11.
- [2] Andrew Graham. Using smartphones in lift travel analysis. Bachelor's thesis, University of Sheffield, 2018.
- [3] Peters Research Ltd. Peters research: Lift traffic analysis software, 2024. Accessed: 2024-09-05.
- [4] Wes McKinney. Python pandas. <https://pandas.pydata.org/>, 2008. Accessed: 2024-09-11.
- [5] Jacob Murel and Eda Kavlakoglu. What is ridge regression? <https://www.ibm.com/topics/ridge-regression>, 2023. Accessed: 2024-09-05.
- [6] Guido van Rossum. Python. <https://www.python.org/>, 1991. Accessed: 2024-09-11.